

Übung 5: Software- und Architekturmetriken für Codequalität und Architekturoptimierung

Team 6 (Thema E-Vote) - Abgabe:

Link: https://github.com/jacque-schu/MSE_eVote

Gemeinsam als Team haben wir uns über Discord zum Thema „Software- und Architekturmetriken für Codequalität und Architekturoptimierung“ ausgetauscht. Hier tragen wir gemeinsam unsere Antworten zusammen.

Aufgabe 1: Überblick und Anwendung einfacher Metriken:

Für Aufgabe 1 haben wir uns gemeinsam ausgetauscht, welches Tool wir für die Erstellung der Metriken verwenden. Dafür testeten wir verschiedene Plugins wie z.B. Code Metrics für unsere IDE PyCharm. Leider mussten wir schnell feststellen, dass die meisten hier im Kurs benannten oder verwendeten Tools für diese Aufgabe für Java ausgelegt sind. Da unser Projekt mit Python programmiert ist/wird, entschieden wir uns letztendlich für SonarQube. Wir nutzten zum Start die Website und bindeten dort unserer Repository ein:

Schritte zur erfolgreichen Integration

- Projekt auf SonarCloud anlegen und mit dem GitHub Repo verbinden
- Dies ermöglicht, dass alle Commits und Pull Requests automatisch analysiert werden.
- Erstellung der Datei sonar-project.properties im Repository-Root: Hier werden Projektschlüssel, Organisation, Source-/Test-Pfade und weitere Einstellungen zentral hinterlegt

Konfiguration der GitHub Actions Pipeline

- Coverage-Bericht (coverage.xml) wird nach Testausführung erzeugt (z.B. mit pytest-cov)
- SonarCloud-Scan wird anschließend als eigener Schritt gestartet

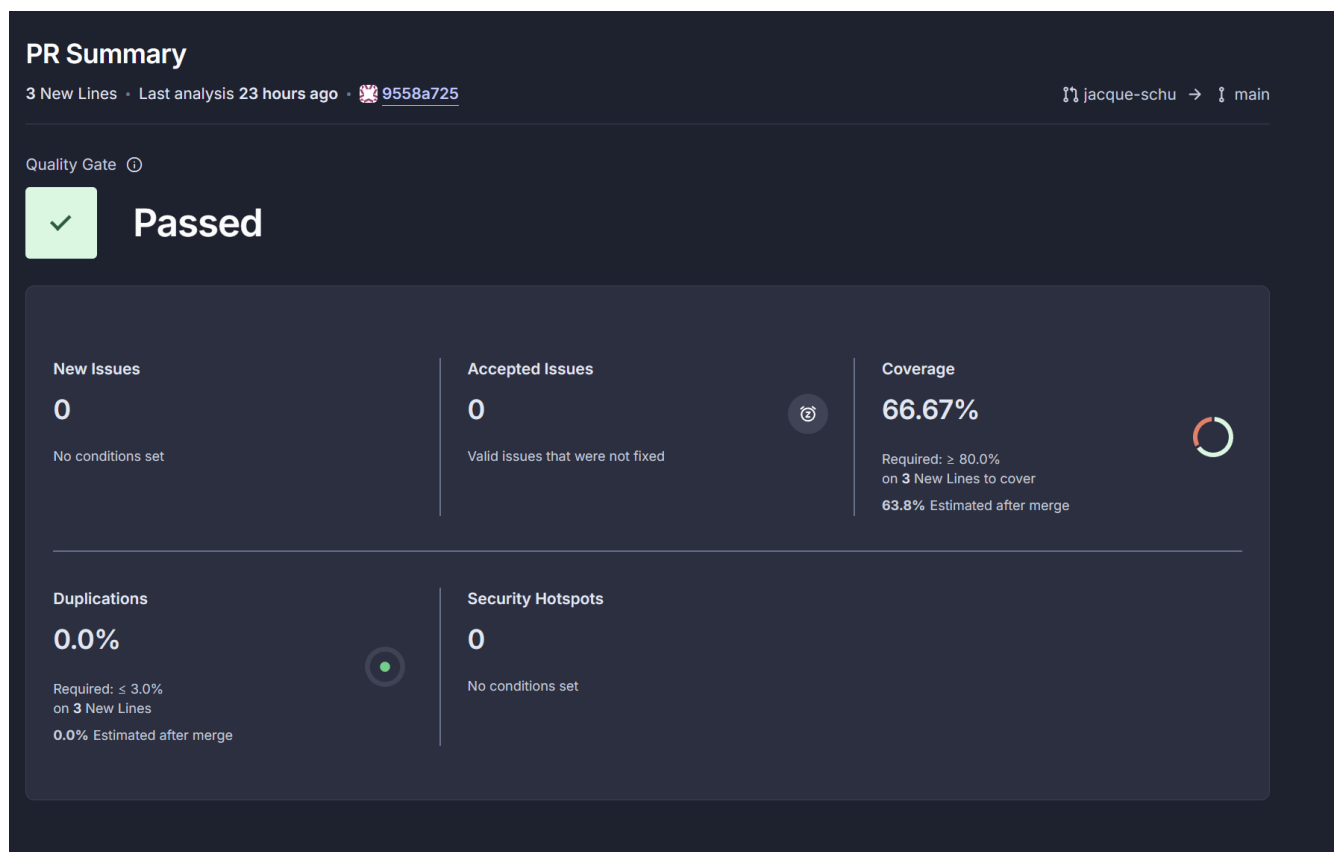
SonarCloud-Token generieren und als Secret in GitHub speichern

- Ein persönlicher Token von SonarCloud wird als Secret SONAR_TOKEN im GitHub-Repository hinterlegt, damit die Analyse authentisiert abläuft.

Vorteile dieser Maßnahme

- Transparenz: Codequalität, Testabdeckung und Schwachstellen werden kontinuierlich sichtbar und nachvollziehbar.
- Schnelles Feedback: Fehler und Verbesserungen werden früh erkannt und können gezielt angegangen werden.
- Automatisierung: Die Pipeline prüft automatisch bei jedem Push, so dass manuelle CodeReviews effizienter werden.
- Teamorientierung: Alle Teammitglieder profitieren von konsistenten Qualitätsstandards und automatischem PR-Feedback.

Das Ergebnis war eine Übersicht im Web durch SonarQube zu unter anderem folgenden Metriken vom letzten PR:



Aufgabe 2: Test Coverage erweitern und Code Coverage verbessern

Diese Aufgabe ist noch fortlaufend und wird nachfolgend ergänzt, da wir zum Zeitpunkt des Schreibens dieses Dokuments (16.11.2025) noch in größeren, aktiven Änderungen im Code arbeiten. Wir werden dies schnellstmöglich, spätestens aber im nächsten Aufgabenblatt ergänzen.

Aufgabe 3: Technical Debt und Regelverletzungen mit LLM analysieren

Für die Analyse wurde wie bereits beschrieben zum einen SonarQube als Plugin in Pycharm installiert und zum anderen das GitHub-Projekt mit der SonarQube Cloud verbunden.

Die Analyse durch SonarQube hat 14 Issues (Stand 17.11.25) ergeben, die mit einem Aufwand von 1h 23min behoben werden können. Die 14 Issues werden kategorisiert und verschiedenen Schweregraden zugeordnet.

Software quality:

- Security (1): Es gibt ein Problem, das die Sicherheit des Codes betrifft (z. B. ein potenzielles Sicherheitsrisiko wie SQL-Injection, unsicherer Umgang mit Authentifizierung).
- Reliability (8): Es gibt acht Probleme, die die Zuverlässigkeit des Codes beeinflussen können (z.B. Fehleranfällige Logik, Ausnahmen, fehlerhafte Bedingungen).
- Maintainability (7): Es gibt sieben Probleme, die in den Bereich der Wartbarkeit fallen, also wie gut der Code gepflegt, erweitert oder gelesen werden kann (z. B. duplizierter Code, schlechte Struktur, zu hohe Komplexität).

Severity:

- Blocker (1): Ein sehr schweres Problem, das sofort behoben werden sollte, da es kritische Fehler, Sicherheit oder Stabilität betrifft.
- High (3): Drei schwerwiegende Probleme, die möglichst schnell behoben werden sollten.
- Medium (11): Elf Probleme mittlerer Wichtigkeit, die zeitnah bearbeitet werden sollten.
- Low (1): Ein leichtes Problem, das weniger dringend behoben werden muss.
- Info (0): Keine reinen Hinweis-Einträge ohne Handlungsbedarf.

Code attribute:

- Consistency (6): Sechs Probleme betreffen die Konsistenz des Codes (uneinheitliche Formatierung, widersprüchliche Namensgebung oder Nutzung verschiedener Methoden für gleiche Aufgaben).

- Intentionality (8): Acht Probleme, bei denen die Absicht im Code nicht klar erkennbar oder nachvollziehbar dokumentiert ist (z. B. unklare Funktionen, fehlende Kommentare oder kryptische Logik).
- Adaptability/Responsibility (0): Keine Probleme bezüglich Anpassungsfähigkeit (wie leicht sich der Code ändern lässt) oder Verantwortlichkeit (wie sauber Zuständigkeiten und Abgrenzungen im Code sind).

Type:

- Bug (6): Es gibt sechs klassische Fehler im Code, die zu falschem Verhalten oder Abstürzen führen können.
- Vulnerability (1): Ein potenzielles Sicherheitsproblem, das z. B. Hackerangriffe oder Datenverlust ermöglichen könnte.
- Code Smell (7): Sieben „Code Smells“ (ineffiziente, wartungsaufwändige oder schlecht lesbare Stellen, die zwar keinen Fehler erzeugen, aber künftige Probleme nach sich ziehen können)

Type Severity:

- Blocker (1): Ein sehr schwerwiegendes Problem, das die Ausführung oder Sicherheit kritisch beeinflusst und sofort gelöst werden sollte.
- Critical (3): Drei kritische Issues, die schnell behandelt werden sollten (können z. B. Ausfälle oder größere Schwachstellen verursachen).
- Major (9): Neun bedeutende aber nicht sofort kritische Probleme.
- Minor (1): Ein kleines oder weniger dringendes Problem.
- Info (0): Keine reinen Hinweise ohne Handlungsbedarf.

The screenshot shows the SonarQube interface with the following filters on the left:

- Software quality:** Security (1), Reliability (8), Maintainability (7)
- Severity:** Blocker (1), High (3), Medium (11), Low (1), Info (0)
- Code attribute:** Consistency (6), Intentionality (8), Adaptability (0), Responsibility (0)
- Type:** Bug (6), Vulnerability (1), Code Smell (7)

The main area displays a list of issues:

- Issue 1:** Delete this unreachable code or refactor the code to make it reachable. (Intentionality, Medium, L29, 5min effort, 4 days ago, Bug, Major)
- Issue 2:** Rename this parameter "buerglerID" to match the regular expression "[a-z][a-z0-9_]*\$". (Consistency, Low, L53, 2min effort, 4 days ago, Code Smell, Minor)
- Issue 3:** Change this code to not reflect unsanitized user-controlled data. (Security, Blocker, L74, 30min effort, 4 days ago, Vulnerability, Major)
- Issue 4:** Fix this call that leads to a attribute access on a value that can be 'None'. (Intentionality, Medium, L146, 10min effort, 1 day ago, Bug, Major)
- Issue 5:** Add a nested comment explaining why this method is empty, or complete the implementation. (Intentionality, High, L16, 5min effort, 4 days ago, Code Smell, Critical)

In SonarQube Cloud ist eine komplette Dokumentation von Issues gegeben. Sie werden kategorisiert, genau benannt, erklärt, warum es sich um eine Regelverletzung handelt, es wird ein Verbesserungsvorschlag aufgezeigt und benannt, wie viel Zeit benötigt wird, die Regelverletzung zu beheben.

Die aufgezeigten Verbesserungsvorschläge können zusätzlich noch mit einem LLM untersucht werden.

Hier ein Beispiel:

Fix this call that leads to a attribute access on a value that can be 'None'. ⓘ

Attributes should not be accessed on "None" values [pythonbugs:S2259](#)

Software qualities impacted: Reliability 🔴 Medium

☐ Open ☐ Not assigned ☒ Bug ☐ Major

Where is the issue? Why is this an issue? **How can I fix it?** Activity More info Open in IDE

If your code contains `if-else` statements or similar constructs where some branches potentially assign the `None` value to a variable, you must ensure that this variable is handled safely afterwards. I.e., its attributes should not be accessed at all or only after explicitly confirming that it is not `None`.

Similarly, for any function calls that can return `None` under certain conditions, carefully confirm that your code avoids these conditions. Again, the safest approach is to check for a `None` return value explicitly.

Noncompliant code example

```
def render(file_path):
    if os.path.isfile(file_path):
        data = interpret_csv(file_path)
    else:
        data = None

    # ...

    data.plot_graph() # Noncompliant
```

Compliant solution

```
def render(file_path):
    if os.path.isfile(file_path):
        data = interpret_csv(file_path)
    else:
        data = None

    # ...

    if data is not None:
        data.plot_graph()
    else:
        print("No data available.")
```

How does this work?

In the given example, the function `render` tries to load information into a variable `data`, depending on whether `file_path` is a path to a file. If this is not the case, `None` is assigned to `data`.

At the end of the function, a method `plot_graph()` is called on `data`. The call is sure to fail if `data` was assigned to `None`. This is prevented by checking first whether `data` is not `None` before performing the call.

Das LLM Perplexity pro hat dazu folgende Antwort:

Das Issue und der Lösungsvorschlag auf dem Screenshot zeigen ein typisches Beispiel dafür, wie SonarQube auf ein konkretes Problem im Code aufmerksam macht und zugleich eine sinnvolle Lösung anbietet.

Bedeutung des SonarQube-Vorschlags

- Problem (Issue):
Die Methode versucht, auf ein Attribut oder eine Methode eines Objekts

(data.plot_graph()) zuzugreifen, das möglicherweise den Wert None hat. Das führt in Python zu einem Laufzeitfehler (AttributeError).

SonarQube erkennt dieses Risiko und meldet eine Verletzung bzgl. Zuverlässigkeit („Reliability“, Medium Severity).

- Nicht-konformer Code:
Der alte Code greift ungeprüft auf das Objekt zu:

```
data.plot_graph() # Noncompliant
```

Falls data vorher None ist, gibt es einen Fehler.

- Konformer Lösungsvorschlag:
Die von SonarQube empfohlene Änderung prüft, ob data nicht None ist, bevor die Methode aufgerufen wird:

if data is not None:

```
    data.plot_graph()
```

else:

```
    print("No data available.")
```

So wird der Laufzeitfehler sauber vermieden.

- Erklärung:
SonarQube liefert im unteren Bereich eine verständliche Erklärung, warum diese Änderung notwendig ist und wie sie funktioniert. Dadurch wird der Code robuster und weniger fehleranfällig.

Fazit

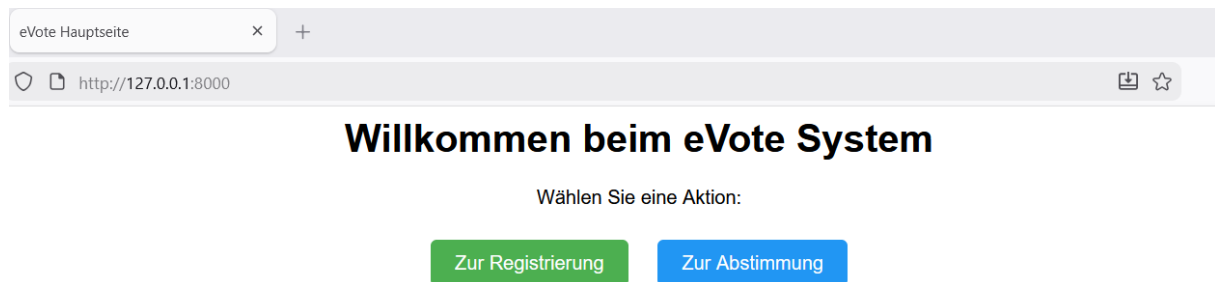
Der gezeigte SonarQube-Vorschlag ist absolut sinnvoll:

- Er entspricht den Best Practices für Python und andere Programmiersprachen.
- Das Problem wird korrekt erkannt und die Lösung ist direkt anwendbar.
- Auch die Zusatzinfos („Effort“, „Line affected“, „Tags“) machen die Umsetzung einfach planbar.

Solche Vorschläge von SonarQube verbessern aktiv die Codequalität und sind empfehlenswert umzusetzen, sofern sie zum Kontext und zur Situation im Projekt passen.

Aufgabe 4: Frontend-Entwicklung und Erweiterung der Anwendung

Im weiteren Verlauf wurde für die Aufgabe 4 Frontend-Entwicklung und Erweiterung der Anwendung eine soweit noch simple, aber leicht anpassbare Maske/Startseite erstellt, die sowohl zur Registrierung als auch zur Abstimmung führt. Nach Auswahl „Zur Registrierung“ gelangt man zu einem Formular, um sich zu registrieren. Dies bildet aktuell die Grundlage für weitere Entwicklungen, die wir in sehr naher Zeit vornehmen werden.



Aufgabe 5: Reflexion zum Einsatz von Metriken und LLM

Der gezielte Einsatz eines LLM hat allgemein unseren Entwicklungsprozess spürbar beschleunigt und qualitativ verbessert, weil aus Analyseergebnissen unmittelbar umsetzbare Schritte wurden. Der größte Mehrwert lag in der Übersetzung von Befunden in klare Maßnahmen für Refactoring und Testentwurf wodurch die Umsetzung konsistenter und zielgerichteter erfolgte.

Konkret half das LLM, aus auffälligen Stellen im Code sinnvolle Vereinfachungen und Modularisierungen abzuleiten, ohne lange Iterationen in der Ideensuche, was die Kluft zwischen Diagnose und Änderung merklich verkleinerte. Für die Qualitätssicherung lieferte das LLM systematische Testideen und Edge Cases für komplexe Funktionen, die die Abdeckung zielgerichtet erhöhten und Regressionen vorbeugten.

Wichtig war dennoch eine kritische Prüfung: Vorschläge des LLM wurden nur übernommen, durch Tests belegbar waren und zu bestehenden Architekturentscheidungen passten. So entstand ein produktiver Arbeitsmodus: Erkenntnisse strukturieren, das LLM für Alternativen und Edge Cases nutzen, Änderungen minimalinvasiv umsetzen und sofort über Reviews und Tests absichern. Insgesamt hat das LLM die Kognitionslast reduziert, Feedbackzyklen verkürzt und das Verständnis über Code vertieft.

Dennoch ist es uns gemeinsam als Gruppe teils schwergefallen, die schier Masse an Code zu den jeweiligen Aufgaben jederzeit selber zu verstehen, damit jedes Teammitglied über jede Aufgabe bestmöglich informiert ist.